General Update of Progress:

A nonlinear dual-track vehicle model has been developed in MATLAB. A Car.m class defines vehicle parameters and is given to the Model_v4() function:

```
% % Inherits Car class. Add car presets here
% % 11/7: "sedan" preset added

classdef CarWithDefaults < Car
    methods
        function obj = CarWithDefaults(type)
            arguments
                type {mustBeMember(type,["sedan","custom"])} = "sedan"
            end

            obj@Car(); % call parent constructor

            if type == "sedan"
                obj = obj.setSedanDefaults();
            end
        end
    end

    methods (Access=private)
        function obj = setSedanDefaults(obj)

            % Pacejka
            obj.pacejka.long.B  = 10 * ones(4,1);
            obj.pacejka.long.C  = 1.9 * ones(4,1);
            obj.pacejka.long.E  = 0.97 * ones(4,1);
            obj.pacejka.long.mu = 0.9 * ones(4,1);

            obj.pacejka.lat.B   = 8 * ones(4,1);
            obj.pacejka.lat.C   = 1.3 * ones(4,1);
            obj.pacejka.lat.E   = 1.0 * ones(4,1);
            obj.pacejka.lat.mu  = 0.9 * ones(4,1);

            % Geometry
            obj.geometry.a       = 1.50;
            obj.geometry.b       = 1.20;
```

```
% Geometry
obj.geometry.a       = 1.50;
obj.geometry.b       = 1.20;
obj.geometry.h       = 0.55;
obj.geometry.t       = 1.60;
obj.geometry.r_wheel = 0.32;

% Dynamics
obj.dynamics.lambda_front = 0.45;
obj.dynamics.I_1 = 1.2;
obj.dynamics.I_2 = 1.2;
obj.dynamics.I_3 = 1.2;
obj.dynamics.I_4 = 1.2;

obj.dynamics.B_1 = 0.01;
obj.dynamics.B_2 = 0.01;
obj.dynamics.B_3 = 0.01;
obj.dynamics.B_4 = 0.01;

obj.dynamics.I_zz     = 2400;
obj.dynamics.m_vehicle = 1500;

obj.dynamics.F_z1_static = 3500;
obj.dynamics.F_z2_static = 3500;
obj.dynamics.F_z3_static = 3500;
obj.dynamics.F_z4_static = 3500;

% Aero
obj.aero.rho_air   = 1.225;
obj.aero.C_drag    = 0.28;
obj.aero.A_frontal = 2.2;
obj.aero.A_side    = 2.5;
```

Passing the Car.m instance to Model_v4() returns the nonlinear f(x,u), A and B symbolic representations for the system where:

$$\dot{\boldsymbol{x}} = f(\boldsymbol{x}, u)$$

$$A = jacobian(f(\boldsymbol{x}, u), \boldsymbol{x})$$

$$B = jacobian(f(\boldsymbol{x}, u), u)$$

And:

$$\boldsymbol{x} = [vx, vy\ , r, \omega_{left}, \omega_{right}]$$

$$u = [T_{motor,left}, T_{motor,right}, \delta]$$

State vector consists of:

$$vx \rightarrow \ Vehicle\ frame\ longitudinal\ velocity$$

$$vy \rightarrow \ Vehicle\ frame\ lateral\ velocity$$

$$r \rightarrow \ Vehicle\ yaw\ rate\ \left(r = \dot{\psi}\right)$$

$$\omega_{left} \rightarrow \ left\ rear\ (wheel\ 2)\ angular\ velocity$$

$$\omega_{right} \rightarrow \ right\ rear\ (wheel\ 3)\ angular\ velocity$$

The function which builds the model is shown below.

```
function Model = Model_v4(car)
disp('starting:')
     [State, StateDeriv, Input, InputDeriv] = makeSymbols2();
        Model.symbols.State = State;
        Model.symbols.StateDeriv = StateDeriv;
        Model.symbols.Input = Input;
        Model.symbols.InputDeriv = InputDeriv;
                    disp('makeKinematics:')
                    [Slip, FrontWheels, wc] = makeKinematics(State, StateDeriv, Input, InputDeriv, car);
                                            disp('makeNormalForces:')
                                            Fz = makeNormalForces(StateDeriv, car);
                                            disp('makeWheelFrameForces:')
                                            WheelFrameForces = makeWheelFrameForces(Slip, car, Fz, FrontWheels);
                                            disp('makeDriveTrain:')
                                            [omega_dot_2, omega_dot_3] = makeDriveTrain(State, Input, car, WheelFrameForces);
                                            disp('makeBodyFrameDynamics2:')
                                            [v_dot_x, v_dot_y, r_dot] = makeBodyFrameDynamics3(car, State, StateDeriv, Input, WheelFrameForces, wc);
                                            f = DAE2ODE(StateDeriv, v_dot_x, v_dot_y, r_dot, omega_dot_2, omega_dot_3);
                                             % Return struct
                                             Model.f = f;
                                             Model.A = getJacobian(f,State);
                                             Model.B = getJacobian(f,Input);

end
```

To linearize the system at an operating point, matlabFunctions were made for A, B and f which evaluate them at an operating point x_op, u_op:

```
% Continuous time A, B, f at operating point
A_fun = matlabFunction(model.A,'Vars',{x_op,u_op});
B_fun = matlabFunction(model.B,'Vars',{x_op,u_op});
f_fun = matlabFunction(model.f,'Vars',{x_op,u_op});
```

A function was created (F_eul) which completes a forward-euler discretization of the continuous A, B, f at the operating point x_op, u_op. The return variable (x_next) is the next discrete state of the system.

```
function x_next = F_eul(x_op,u_op,Ts,A_fun,B_fun,f_fun)

Ac_lin = A_fun(x_op,u_op);
Bc_linn = B_fun(x_op,u_op);
Bc_lin = [Bc_linn,zeros(5,1)];   %delta_dot is small.
fc_lin = f_fun(x_op,u_op);
I = eye(5);

% change column vectors to vertical
x_op = x_op';
u_op = u_op';
%
x_now = x_op;
u_now = u_op;

    % Forward Euler step with affine correction term
    x_next = (I + Ts*Ac_lin)*x_now + Ts*Bc_lin*u_now + Ts*(fc_lin-Ac_lin*x_op-Bc_lin*u_op);
end
```

Discretization was resolved using a numerical algorithm as well. CasADi framework was used with mpcTools to solve for the next discrete state given the current state and input using numerical integration with runge-kutta. The function F_rk() returns the next state of the system (in the same way as F_fe()).

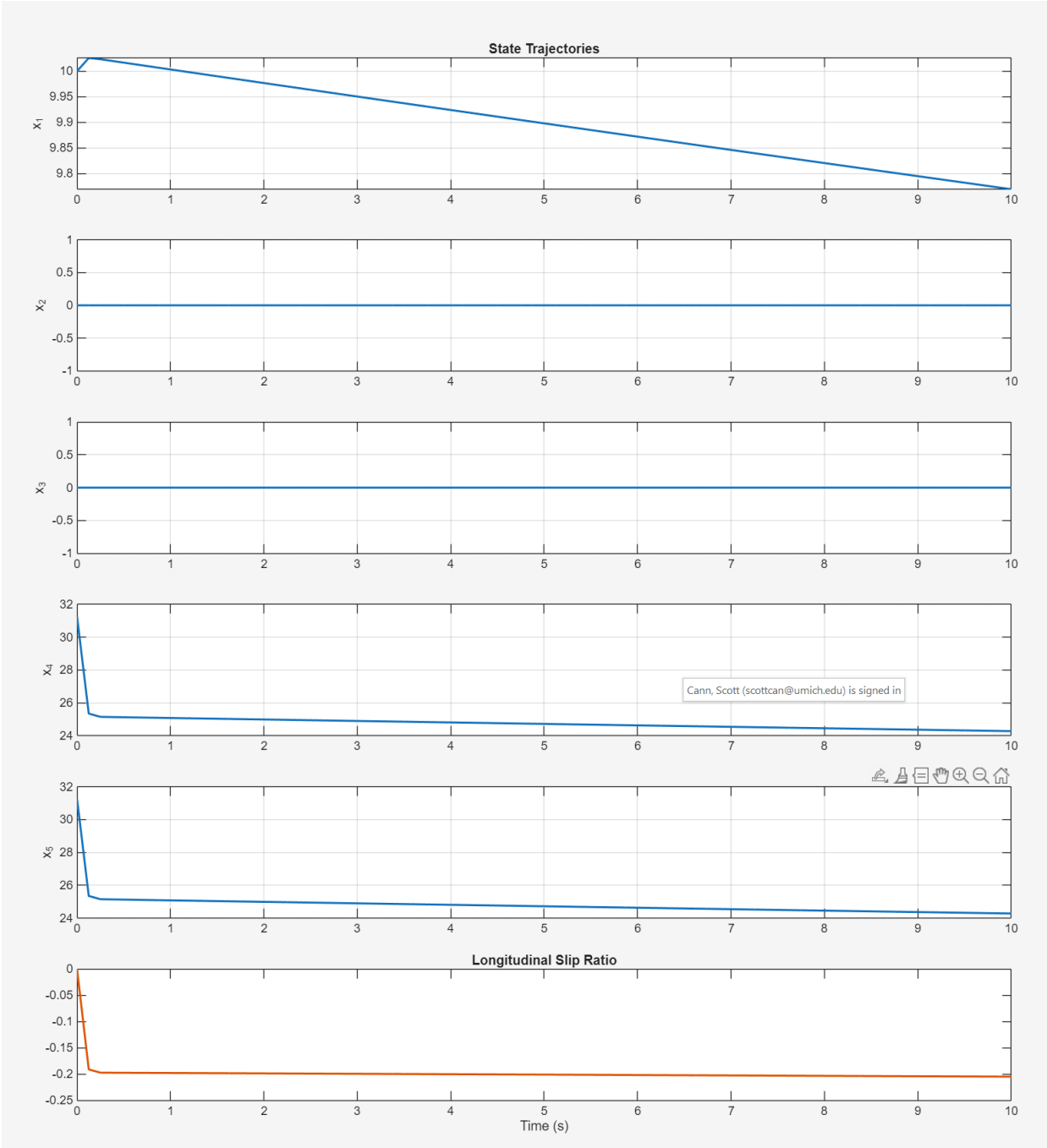
```
% Setup time-integration method. Discretizes the system, numerical
% framework to compute xk+1 from xk, uk. Uses more accurate methods than
% simple forward Euler (1st degree Taylor) discretization

% Integrator to discretize system
intg_options = struct();
intg_options.tf = Ts;
intg_options.simplify = true;
intg_options.number_of_finite_elements = 4;

% Pass ODEs to Integrator (can also be DAEs. Wish I knew this earlier. lol)
dae = struct();
dae.x = x_cas;  %states
dae.p = u_cas;  %parameters (fixed during integration horizon, ie Ts, ZOH)
dae.ode = f_cas(x_cas,u_cas);   %expression for rhs

% Build integrator for discretization, runge-kutta
intg = integrator('intg','rk',dae,intg_options);    % make rk integrator
res = intg('x0',x_cas,'p',u_cas);   % make function for evaluating @ x,u
x_next = res.xf;    % Now return only last element on integration horiz.
F_rk = Function('F_rk',{x_cas,u_cas},{x_next},{'x','u'},{'x_next'});  %wrap, F(x_op,u_op) returns x_k+1
```

To make sure the system behavior is reasonable, the vehicle was simulated for 10 seconds with no input and coasting from 10m/s and no wheel slip or steering angles. A Runge-Kutta numerical algorithm was used to resolve the evolution of system dynamics.



Implementation challenges:

1. I believe I made my system more complex than was helpful to developing a model-predictive controller. It took a very long time to get to this point. I am attempting to implement NMPC using CasADi + mpcTools. I found some good resources, and it is setup in MATLAB.

2. The issue I am running into right now is this: I have the nonlinear system x_dot = **f**(x,u), where **f** is defined with *matlab symbolic* toolbox. In order to use CasADi NLQP solver I need to convert this to Casadi's own symbolic framework. I can solve the autonomous system using the CasADi framework, but cannot give it inputs and implement MPC at the moment. I may have to rework my model to output CasADi symbolic, as manually hard-coding is not possible as there are a very large number of terms.

Plan of Activities for the rest of project:

1. Complete the implementation of CasADi NMPC and MPC solvers
2. Test ability of both solvers to control vehicle velocity and yaw rate (turning rate) control of vehicle, when in both wet and dry conditions (low mu, high mu, peak tractive force in pacejka formulation)
3. The original goal of testing different traction conditions on the left and right side of the vehicle is unfeasible in the time remaining, so instead I plan to test the linear and nonlinear controller performance while driving in wet and dry conditions.